

When Does Chain-of-Thought Help? A Controlled Study of Prompting Strategies on Reasoning and Classification Tasks

Amirhossein Khishvand
Sapienza University of Rome
MSc Course: Computer Science

Abstract

Chain-of-Thought (CoT) prompting is often reported to improve the reasoning ability of large language models, but it is not free: it makes the model produce many more tokens, which costs time and money. This project asks a simple question – *when* does CoT actually help? – through a small and controlled set of experiments. Using a single open-weight instruction-tuned model, I compare three prompting strategies (direct answering, zero-shot CoT, and few-shot CoT) on a multi-step arithmetic reasoning task (GSM8K) and a shallow sentiment classification task (SST-2). My hypothesis is that CoT gives a large gain on the reasoning task but almost no gain on the classification task. As a second experiment, I test self-consistency (sampling several reasoning paths and taking a majority vote) to see whether it improves reasoning accuracy further. The goal is not a new method but a clean and reproducible demonstration of when CoT is, and is not, worth its cost.

1 Task description/Problem statement

The task studied here is **prompting strategies for large language models**, and in particular Chain-of-Thought (CoT) prompting [7]. Given a natural-language question, the model must produce an answer; the variable of interest is *how* we prompt it – whether we ask for the answer directly, or ask it to reason step by step first. CoT prompting elicits intermediate reasoning steps before the final answer, either by prepending worked examples (few-shot CoT) or by adding a trigger phrase such as “Let’s think step by step” (zero-shot CoT) [2]. I treat this as an evaluation problem: I measure how much each prompting strategy changes accuracy on two

different kinds of task, one that needs multi-step reasoning and one that does not. The central hypothesis is that CoT helps mostly where intermediate computation is required.

1.1 Examples

Two tasks are used, so there are two input/output formats.

GSM8K (reasoning).

- *Input*: a math word problem, e.g. “Natalia sold 48 clips in April and half as many in May. How many did she sell in total?”
- *Expected output (direct)*: “The answer is 72.”
- *Expected output (CoT)*: “May = $48/2 = 24$. Total = $48 + 24 = 72$. The answer is 72.”

SST-2 (classification).

- *Input*: a short movie-review sentence, e.g. “a warm, funny, and deeply moving triumph.”
- *Expected output*: a single label, “positive” or “negative.”

1.2 Real-world applications

Understanding when CoT helps has direct practical value. In real deployments, prompting a model to reason step by step multiplies the number of generated tokens, which increases latency and cost. Knowing which tasks genuinely benefit lets practitioners apply CoT only where it pays off (e.g. math, multi-hop question answering, code reasoning, planning) and avoid it on shallow tasks (e.g. sentiment, topic labeling, simple retrieval) where a direct answer is just as accurate but far cheaper. The same reasoning applies to self-consistency, which multiplies cost even further by sampling several chains per question.

2 Related work

This project builds on three well-known works rather than proposing something new. Chain-of-Thought prompting was introduced by Wei et al.[7], who showed that asking a model to produce intermediate reasoning steps improves performance on arithmetic, commonsense, and symbolic tasks. Kojima et al.[2] showed that CoT can also be triggered zero-shot with a simple phrase, without hand-written examples. Wang et al.[6] introduced self-consistency, which replaces greedy decoding of one chain with sampling several chains and taking the majority-voted answer, giving consistent gains on reasoning benchmarks. These three define the conditions I compare. The state of the art on GSM8K has since moved to much larger models and more advanced reasoning methods, but the core finding – that reasoning steps help most on multi-step problems – is exactly what this project examines in a controlled way.

3 Datasets and benchmarks

Two standard benchmarks are used, chosen so that one needs reasoning and the other does not.

- **GSM8K** [1]: grade-school math word problems that need several arithmetic steps. Gold answers come in a fixed “#### (number)” format (for example, “#### 72”), which makes exact-match scoring easy.
- **SST-2** [4] (from the GLUE benchmark [5]): binary sentiment classification of short movie-review sentences, a shallow task with no multi-step computation.

For each task I use a random subset of 50 examples per condition, drawn with a fixed random seed for reproducibility.

4 Existing tools, libraries, papers with code

The whole pipeline is built on public, open-source tools. Datasets are loaded directly from the Hugging Face datasets library (`openai/gsm8k`, `nyu-mll/glue`). Inference uses the `transformers` library [8] with the open-weight model `Qwen2.5-7B-Instruct` [3], loaded in 4-bit precision via `bitsandbytes` so it fits on a single small GPU. Plotting uses `matplotlib`. CoT and self-consistency are implemented directly from the original papers rather

than a single reference repository, keeping the code small and easy to check. The methods have public reference code associated with their papers, and the datasets and model are all openly available on the Hugging Face Hub.

5 State-of-the-art evaluation

In the literature, prompting strategies on these tasks are evaluated with **exact-match accuracy**: the model’s final answer is extracted and compared to the gold answer. On GSM8K, the final number is parsed from the generated text; on classification tasks like SST-2, the predicted label is compared to the gold label. State-of-the-art systems report accuracy on the full test sets and often average over several runs or seeds to account for sampling variance. I follow the same exact-match protocol on sampled subsets, which is standard for small-scale controlled studies.

6 Comparative evaluation

This is the comparative part of the project (HWp).

- **Datasets:** GSM8K (reasoning) and SST-2 (classification), 50 examples per task per condition, fixed seed.
- **Systems and baseline:** a single fixed model (`Qwen2.5-7B-Instruct`) is compared across four strategies. Direct answering is the *baseline*; zero-shot CoT, few-shot CoT, and self-consistency are the methods compared against it. Self-consistency is applied only to GSM8K, since it is a reasoning-aggregation method and offers nothing to vote over on a shallow binary task.
- **Measures and protocol:** exact-match accuracy. Experiment 1 (direct / zero-shot CoT / few-shot CoT) uses greedy decoding, so it is deterministic. Experiment 2 (self-consistency) fixes the prompt to zero-shot CoT and samples k reasoning paths at temperature $T = 0.7$, taking the majority-voted answer:

$$\hat{a} = \arg \max_a \sum_{i=1}^k \mathbb{1}[a_i = a], \quad (1)$$

the most frequent answer across the k paths ($k = 5$).

Condition	GSM8K	SST-2
Direct	0.160	0.940
Zero-shot CoT	0.920	0.920
Few-shot CoT	0.860	0.940
Self-consistency ($k=5$)	0.680	—

Table 1: Exact-match accuracy by task and prompting condition ($N = 50$ per task per condition).

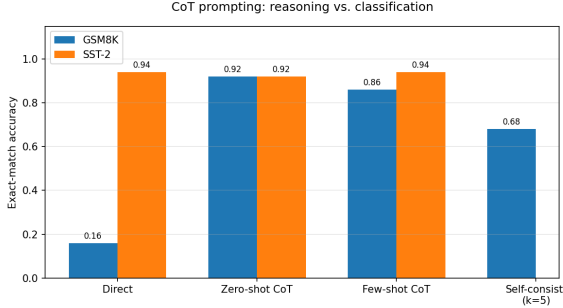


Figure 1: Exact-match accuracy by task and prompting condition. Self-consistency is reported for GSM8K only.

Answer extraction for GSM8K uses a regular expression that first looks for an explicit “the answer is $\langle n \rangle$ ” cue and falls back to the last number on the final line, stripping LaTeX wrappers such as `\boxed{}`. The extraction was checked on a sample of outputs before the full run. Experiments ran on a single NVIDIA T4 (16 GB) on Lightning AI; the pipeline was first debugged on CPU with a tiny model (Qwen2.5-0.5B-Instruct) to avoid wasting GPU time on bugs.

6.1 Results

Table 1 and Fig. 1 report exact-match accuracy per task and per condition.

The main result is the clear contrast between the two tasks. On GSM8K, direct answering is very weak (0.160): without reasoning steps, the model rarely reaches the correct final number. Adding CoT raises accuracy sharply, to 0.920 (zero-shot) and 0.860 (few-shot) – a gain of about 0.70 in absolute terms. On SST-2, all three conditions stay high and almost flat (0.940 / 0.920 / 0.940): reasoning steps add nothing, because the task can be solved in one step. This supports the hypothesis that CoT’s value depends on whether the task actually needs intermediate computation.

6.2 Discussion

Few-shot CoT did not beat zero-shot CoT on GSM8K; the simple “let’s think step by step” trigger was already enough to elicit good reasoning, and the worked examples gave no extra benefit (they were even slightly lower, within the noise expected at $N = 50$).

Self-consistency gave a more surprising result: at $k = 5$ it scored 0.680 on GSM8K, *below* both single-chain CoT conditions, against the expectation that majority voting improves reasoning. I attribute this mainly to two factors. First, the sampled chains used a shorter maximum generation length than the greedy CoT runs, so some temperature-0.7 samples – which tend to reason at greater length – were cut off before stating a final answer, causing extraction to miss or misread the result. Second, sampling variance on a 50-example subset is large. The takeaway is that self-consistency’s benefit depends on each sampled chain being allowed to finish, and that the method is sensitive to generation budget and sample size; with a matched budget and more examples I would expect it to at least match single-chain CoT.

More broadly, the limits of this comparison come from its small scale: one model, one run per condition, and 50-example subsets rather than full test sets. The committed errors on GSM8K are mostly genuine reasoning or arithmetic mistakes by the model, plus a smaller number of extraction failures on truncated chains; on SST-2 the few errors are on genuinely ambiguous reviews. Greedy decoding in Experiment 1 removes sampling noise but also removes any measure of run-to-run variation.

7 Conclusions

This project is a small, reproducible study of when Chain-of-Thought prompting is worth its cost. By changing only the prompting strategy while keeping the model and data fixed, and by contrasting a reasoning task with a classification task, it shows that CoT’s benefit is concentrated where multi-step computation is needed: CoT lifted GSM8K accuracy from 0.160 to 0.920 while leaving SST-2 almost unchanged. The self-consistency experiment did not improve over single-chain CoT in my setup, which I trace to a generation-budget mismatch and small-sample variance rather than a failure of the method itself. **Future work:** re-run

self-consistency with a generation budget matched to the greedy runs, sweep k , use larger subsets (or full test sets) with multiple seeds for confidence intervals, and add a second model size to test whether the CoT gain grows with scale.

Use of Generative AI Tools. I used an AI assistant (a large language model) in a few supporting roles: to polish the wording of this report, to generate the plotting code for the results chart, and to produce the initial skeleton/structure of the experiment code. Everything the AI produced was supervised and carefully checked by me, so that the logic, the experimental structure, and the code are correct and free of bugs. All the ideas – the research question, the choice of tasks and comparison, and the way results from the different papers are combined and interpreted – are my own work.

Contributions. This project was carried out by a single author.

References

- [1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 2
- [2] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2
- [3] Qwen Team. Qwen2.5 technical report, 2024. Qwen2.5-7B-Instruct model, available on the Hugging Face Hub. 2
- [4] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013. 2
- [5] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2018. 2
- [6] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. 2
- [7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2
- [8] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020. 2